

VERIQTA

Built by
Engineers
for Engineers

AWS

Think
Investigate
Resolve
Prevent

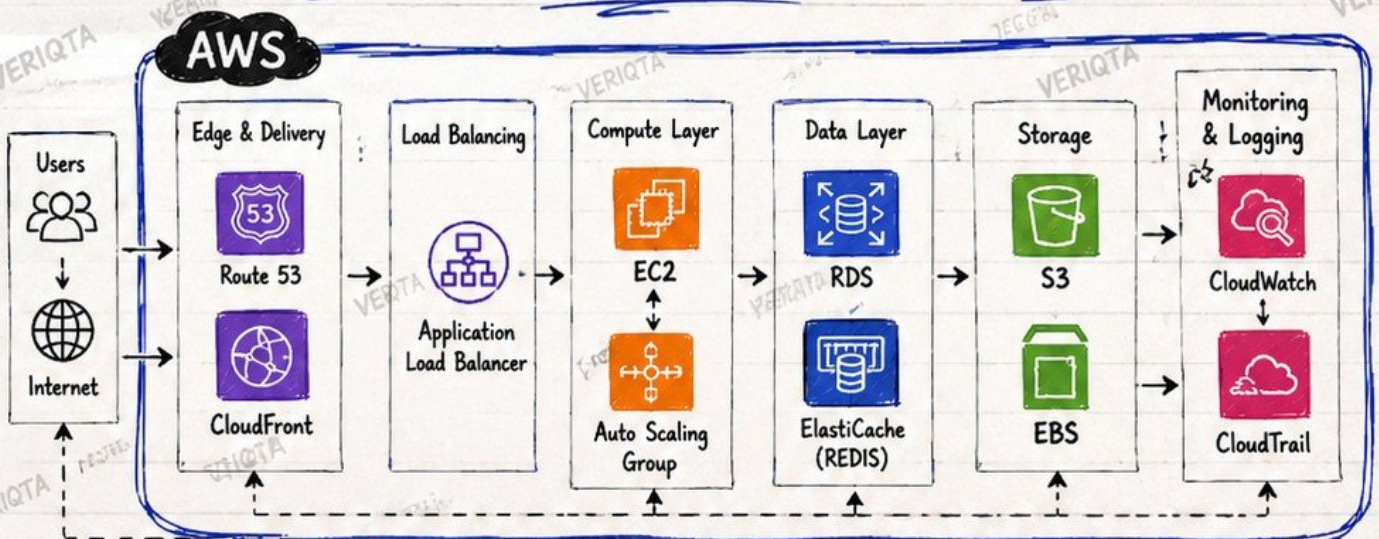
PRODUCTION INCIDENTS

VOLUME 1

From
Outage
To
Solution

Real-world failures, investigations,
and production best practices

Think
Like a
Senior
Engineer



REAL INCIDENTS

15 critical AWS
production issues
engineers face daily



INVESTIGATE

Step-by-step
investigation
workflows



RESOLVE

Practical fixes
that work in
production



PREVENT

Best practices to
build resilient
AWS systems



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA
ENGINEERING
EXCELLENCE

INCIDENT #1

EC2 INSTANCE SUDDENLY UNREACHABLE



★ One moment your app is running fine, the next minute the EC2 instance is **unreachable**. Users can't connect, SSH fails, apps down.

PRODUCTION SCENARIO

Your application hosted on an EC2 instance was working fine. Suddenly, users start reporting downtime. You can't SSH into the instance and the app is not responding.



SYMPTOMS

- ✗ Cannot SSH into the instance
- ✗ Application is not accessible
- ✗ Health checks failing
- ✗ Instance status shows as running but unreachable
- ✗ Connection timeout errors
- ✗ High error rates in application



SENIOR ENGINEER INVESTIGATION

- ✓ Don't guess. Follow a **systematic** path.
- ✓ Start from the outside (**network**) and move inward.
- ✓ Check AWS health before diving deep.
- ✓ Validate connectivity, security, routes, and instance health.
- ✓ Review recent changes & deployments.
- ✓ Correlate with monitoring & logs.

POSSIBLE ROOT CAUSES



- Security Group blocking SSH/HTTP
- Network ACL blocking traffic
- Incorrect Route Table / No route to Internet
- Instance in a private subnet without NAT/IGW
- Instance stopped or hibernated
- CloudWatch Agent or OS issue
- High CPU / Memory causing OS freeze
- Systemd / Service crash
- Underlying AWS infrastructure issue

INVESTIGATION WORKFLOW

- 1 Check AWS Health Dashboard (Global/Regional)
- 2 Check if instance is running and status checks
- 3 Check Security Group inbound rules
- 4 Check NACL rules
- 5 Check Route Table for the subnet
- 6 Test connectivity (ping/traceroute) from bastion or another instance
- 7 Check instance CPU, Memory, Disk metrics in CloudWatch
- 8 Check system logs on OS
- 9 Review recent changes / deployments
- 10 Identify & fix the root cause

USEFUL COMMANDS (AWS CLI)

```
# Instance status
aws ec2 describe-instances --instance-ids i-1234567890abcdef0

# Status checks
aws ec2 describe-instance-status --instance-ids i-1234567890abcdef0

# Security Group rules
aws ec2 describe-security-groups --group-ids sg-12345678 # Check instance state

# NACL rules
aws ec2 describe-network-acls --network-acl-ids acl-12345678

# Route Table
aws ec2 describe-route-tables --filters "Name=association.subnet-id,\
Values=subnet-12345678"

# Reachability test (from bastion) # Network connectivity test
ping <private-ip>
traceroute <private-ip>

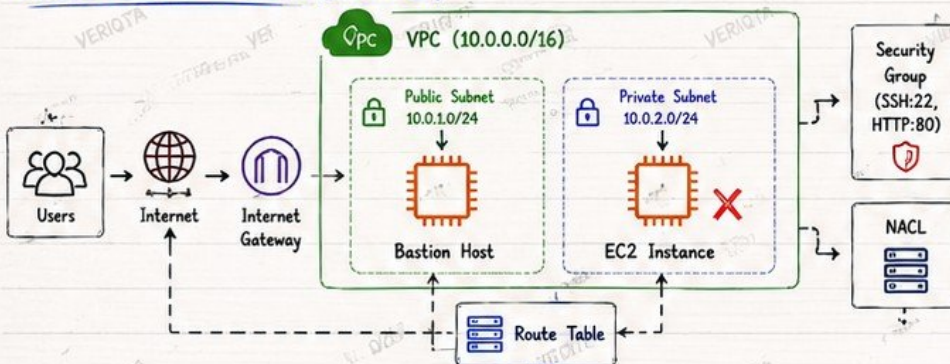
# CloudWatch metrics # Performance metrics
aws cloudwatch get-metric-statistics --namespace AWS/EC2 \
--metric-name CPUUtilization --dimensions Name=InstanceId,Value=i-1234567890abcdef0 \
--start-time $(date -u -d '1 hour ago' +%FT%TZ) \
--end-time $(date -u +%FT%TZ) --period 60 --statistics Average
```

WHAT TO CHECK FIRST?

1. AWS Health Dashboard
2. Instance Status Checks
3. Security Group
4. NACL
5. Route Table
6. Network Connectivity



ARCHITECTURE DIAGRAM (EXAMPLE)



BEST PRACTICES

- ✓ Use least-privilege Security Groups
- ✓ Keep NACLs stateless and simple
- ✓ Use private subnets + NAT for security
- ✓ Enable EC2 Status Check monitoring
- ✓ Set up CloudWatch Alarms for CPU, Network, Disk, Status Check
- ✓ Use Systems Manager (SSM) for secure access without SSH
- ✓ Document network architecture
- ✓ Test recovery procedures regularly
- ✓ Keep OS & applications updated

★ Always investigate **methodically**. Start with the basics (connectivity, security, routes) before assuming the instance is broken.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #2

VERIQTA

EC2 STATUS CHECK FAILED



PRODUCTION SCENARIO

Users report outages. You check EC2 console and see Status Check Failed. Your instance might reboot, become unreachable, or stop responding.



AWS reports that your instance is unhealthy.
The root cause could be AWS infrastructure or your instance.

TWO TYPES OF STATUS CHECKS

1 SYSTEM STATUS CHECK



- ✓ Checks the underlying AWS infrastructure (host, hardware, network, power)
- ✓ If this fails → AWS problem outside your control
- ✓ You may see impairment events in the console

2 INSTANCE STATUS CHECK



- ✓ Checks the health of your instance (OS, software, network stack)
- ✓ If this fails → OS or configuration issue
- ✓ You have more control to fix it

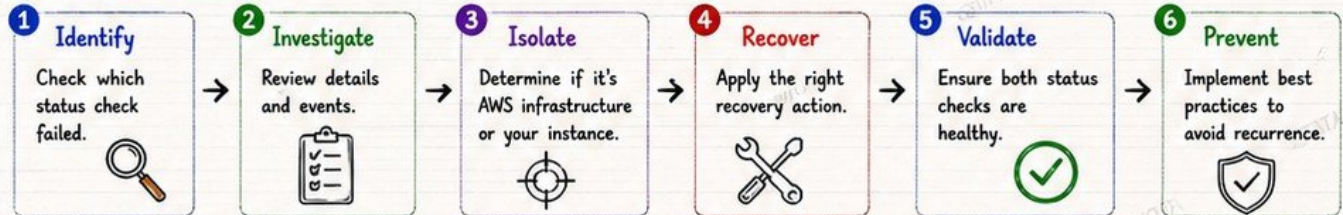
HOW TO QUICKLY IDENTIFY

- ✗ Both checks failing → likely AWS infrastructure issue
- ✗ Instance check failing, system check OK → issue inside your instance

AWS CONSOLE PATH

EC2 Dashboard → Instances → Select Instance → Status Checks Tab

RECOVERY WORKFLOW



WHERE TO SEE DETAILS

Status Checks ⓘ

Status Check	Details
⚠ System Status Check	⚠ Failed
⚠ Instance Status Check	⚠ Failed

- ✗ Click the failed check to see details.
- ✗ If System Check Failed → check AWS Health Dashboard
- ✗ If Instance Check Failed → connect via console/SSM and investigate OS

COMMON CAUSES

SYSTEM STATUS CHECK FAILED

- Host hardware failure
- Network hardware issue
- Power issue in the AZ
- Impaired AWS infrastructure

INSTANCE STATUS CHECK FAILED

- OS crash / kernel panic
- High CPU / Memory exhaustion
- Disk full
- Network stack / firewall issue
- Application deadlock
- Incorrect OS or driver

RECOVERY ACTIONS

IF SYSTEM CHECK FAILED

- ✗ Wait and monitor
- ✗ Check AWS Health Dashboard
- ✗ If persistent → stop and start instance
- ✗ If still failing → use AMI / snapshot to launch new instance

IF INSTANCE CHECK FAILED

- ✗ Connect via EC2 Instance Connect / SSM
- ✗ Check OS logs (dmesg, /var/log)
- ✗ Check disk, memory, CPU, network
- ✗ Restart services / reboot if required

PRODUCTION BEST PRACTICES



Enable Status Check Monitoring (CloudWatch Alarms)



Use Auto Scaling to replace unhealthy instances automatically



Keep AMIs updated and hardened



Distribute across multiple AZs for high availability



Enable detailed monitoring for critical instances



Regularly test recovery and failover



SENIOR ENGINEER TIP

Never assume the problem is in AWS or your instance. Investigate both methodically.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #3

APPLICATION RETURNING 503 ERRORS



PRODUCTION SCENARIO

Users can't access the application. Load Balancer returns 503 Service Unavailable. Some requests might work, others fail. Backend instances may be unhealthy or unreachable.



HTTP 503 from a Load Balancer usually means the service is **UNAVAILABLE** on the backend.

WHAT YOU SEE

- Users get HTTP 503 errors from the Load Balancer.
- ALB/NLB returns 503.
- Target group shows unhealthy or 0 healthy targets.
- CloudWatch alarms for 5XX errors spike.



LOAD BALANCER INVESTIGATION FLOW



1. Check Load Balancer (ALB/NLB)
Is it healthy?



2. Check Target Group
Any healthy targets?



3. Check Health Checks
Are they passing?



4. Check Instance Health
Are instances running?



5. Check Auto Scaling
Are new instances launching?

HEALTH CHECKS



- Verify protocol (HTTP/HTTPS)
- Correct path (ex: /health)
- Correct port
- Healthy threshold
- Unhealthy threshold
- Timeout & Interval

If health checks fail, LB will mark targets **UNHEALTHY** and return 503.



TARGET GROUPS



- Correct targets registered
- All AZs have healthy targets
- Deregistration delay too high?
- Stickiness misconfigured?
- Cross-zone load balancing enabled?

No healthy targets = 503 from Load Balancer.



AUTO SCALING



- Desired capacity vs running capacity
- Scaling policies working?
- Cooldown periods
- Instance launch failures (check Activity History)
- AZ capacity issues

If ASG can't add healthy instances, 503 continues.



COMMON ROOT CAUSES



- All targets unhealthy
- App crash / app not listening
- Health check path incorrect
- Security Group blocking traffic
- Instance CPU/Memory exhausted
- DB or downstream dependency failure
- Auto Scaling not launching new instances

CLI COMMANDS (AWS CLI)



```
# Check target group health
aws elbv2 describe-target-health \
--target-group-arn <target-group-arn>

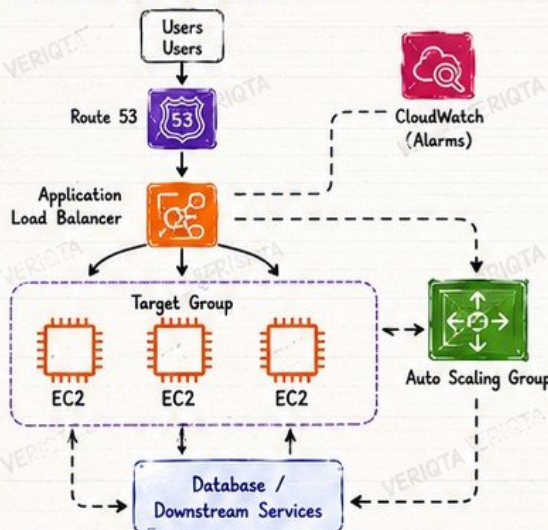
# Check target group description
aws elbv2 describe-target-groups \
--names <target-group-name>

# Check load balancer
aws elbv2 describe-load-balancers

# Check Auto Scaling Group activity
aws autoscaling describe-scaling-activities \
--auto-scaling-group-name <asg-name>

# Check instance health
aws ec2 describe-instance-status \
--instance-ids <instance-id>
```

ARCHITECTURE EXAMPLE



BEST PRACTICES



- Use meaningful health check paths (ex: /health, /ready)
- Set appropriate thresholds and timeouts
- Enable Auto Scaling across multiple AZs
- Monitor 5XX errors and target health
- Use connection draining (deregistration delay)
- Test your recovery by simulating failures

If targets are unhealthy or no instances available, ALB returns 503 Service Unavailable.

503



SENIOR ENGINEER TIP:

Always follow the path: Load Balancer → Target Group → Health Check → Instance → ASG. Find the first unhealthy component and fix the root cause.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #4

VERIQTA

SECURITY GROUP MISCONFIGURATION



PRODUCTION SCENARIO

Users report they can't access the application.
The EC2 instance is healthy, but connections time out.
After investigation, the issue is a Security Group rule.



- ★ Your instance is running, but traffic can't reach it (or too much traffic is allowed in).
A small mistake in Security Groups can cause big production outages.

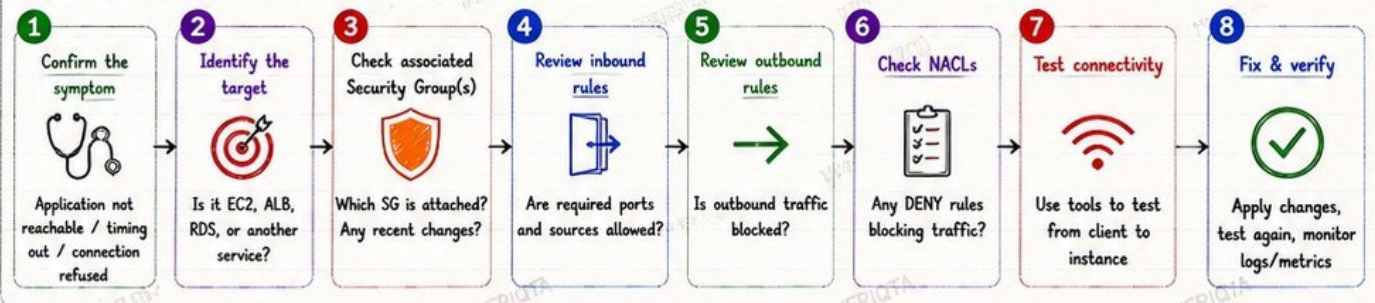
SECURITY GROUPS vs NACLs

	SECURITY GROUPS	NACLs
Type	Stateful (tracks traffic)	Stateless (does not track traffic)
Rule Evaluation	Allow rules only	Allow and Deny rules
Applied To	ENI (Instance level)	Subnet level
Return Traffic	Automatically allowed	Must allow explicitly
Default	Deny all inbound Allow all outbound	Allow all inbound Allow all outbound
Use Case	Control traffic to instances	Control traffic at subnet boundary

COMMON PRODUCTION MISTAKES

- ✗ Opening 0.0.0.0/0 to sensitive ports (22, 3306, 5432, etc)
- ✗ Forgetting to allow the application port (e.g., 80/443/8080)
- ✗ Assuming NACLs are stateful like Security Groups
- ✗ Blocking return traffic (ephemeral ports)
- ✗ Attaching wrong Security Group to the instance/ALB
- ✗ Too many rules hit the limit (60 rules per SG)
- ✗ Allowing all outbound traffic unnecessarily
- ✗ Not considering SG rules of dependent services (RDS, ElastiCache, etc.)

INVESTIGATION WORKFLOW



INEFUL AWS CLI COMMANDS

```
# List security groups
aws ec2 describe-security-groups
```

```
# Show inbound rules of a SG
aws ec2 describe-security-groups \
  --group-ids sg-12345678 \
  --query "SecurityGroups[0].IpPermissions"
```

```
# Show outbound rules of a SG
aws ec2 describe-security-groups \
  --group-ids sg-12345678 \
  --query "SecurityGroups[0].IpPermissionsEgress"
```

```
# Describe network ACLs
aws ec2 describe-network-acls \
  --filters Name=vpc-id,Values=vpc-12345678
```

```
# Describe NACL entries
aws ec2 describe-network-acls \
  --network-acl-ids acl-12345678
```

```
# Test connectivity (from EC2)
telnet <target-ip> <port>
curl -v http://<target-ip>:<port>
nc -vz <target-ip> <port>
```

BEST PRACTICES

- ✓ Use least-privilege: open only required ports
- ✓ Restrict sources to specific IPs or CIDR ranges
- ✓ Use Security Group references (SG-to-SG)
- ✓ Keep SG rules simple and easy to audit
- ✓ Use descriptive names and tags
- ✓ Regularly review and remove unused rules
- ✓ Monitor with VPC Flow Logs and CloudWatch
- ✓ Document network access requirements

SENIOR ENGINEER TIP:

When in doubt, start from the outside and work your way in.
Network issues are often simple: one missing rule can break everything.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #5

VPC CONNECTIVITY FAILURE

VERIQTA



- ★ Your application or instances can't reach the Internet, another subnet, or resources in different AZs/Accounts. It's usually a **routing**, **gateway**, or configuration problem.

PRODUCTION SCENARIO

Users report timeouts. Instances can't reach external services or other internal resources. You suspect a VPC connectivity issue.



ROUTE TABLES



- Route tables control where network traffic goes.
- Each subnet must be associated with the right route table.

Common issues:

- 0.0.0.0/0 route missing for Internet access
- Wrong target (IGW/NAT missing)
- Routes pointing to wrong TGW/VPC peering
- Stale routes after gateway deletion

INTERNET GATEWAY (IGW)



- Allows instances in public subnets to access the Internet.
- Must be attached to the VPC.
- Public subnet route table must have 0.0.0.0/0 → IGW

CHECK

```
aws ec2 describe-internet-gateways
aws ec2 describe-route-tables \
  --filters "Name=route.gateway-id,Values=igw-xxxx"
```

NAT GATEWAY



- Allows instances in private subnets to access the Internet.
- Must be in a public subnet.
- Private subnet route table must have 0.0.0.0/0 → NAT Gateway

CHECK

```
aws ec2 describe-nat-gateways
aws ec2 describe-route-tables \
  --filters "Name=route.nat-gateway-id,Values=nat-xxxx"
```

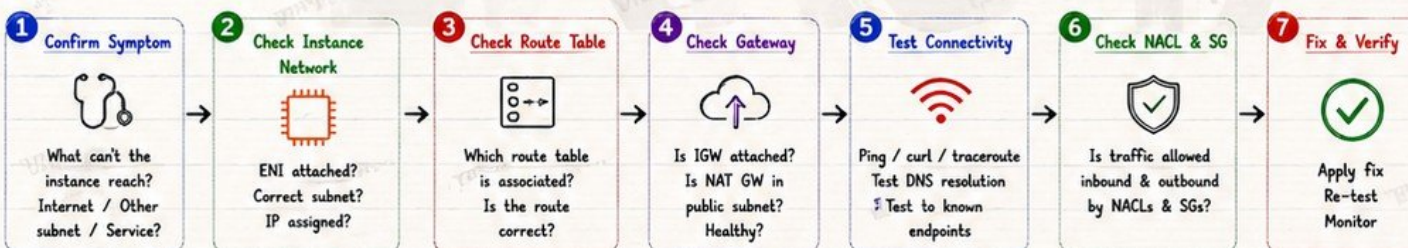
SUBNET TROUBLESHOOTING



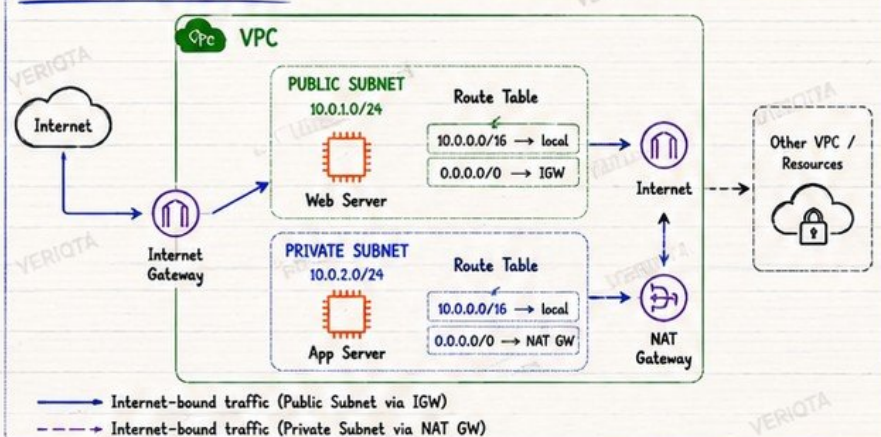
- Is the instance in the correct subnet?
- Is the subnet associated with the correct route table?
- Is the subnet public or private?
- Is the CIDR range overlapping?
- Are network ACLs blocking traffic?
- Is the SG blocking the traffic?
- Is the destination reachable?
- Is DNS resolution working?



INVESTIGATION WORKFLOW



ARCHITECTURE EXAMPLE



USEFUL COMMANDS (AWS CLI)



```
# Describe route tables
aws ec2 describe-route-tables

# Get subnet and its route table
aws ec2 describe-subnets --subnet-ids subnet-xxxx

# Describe Internet Gateways
aws ec2 describe-internet-gateways

# Describe NAT Gateways
aws ec2 describe-nat-gateways

# Test connectivity (from instance)
ping -c 4 google.com
curl -I https://example.com
traceroute 8.8.8.8

# Check security groups
aws ec2 describe-security-groups
```

BEST PRACTICES



- Design clear subnet strategy: Public, Private, Isolated.
- Keep route tables simple. Avoid unnecessary routes.
- Use NAT Gateway in each AZ for high availability.
- Monitor VPC metrics (Reachability Analyzer, Flow Logs, CloudWatch).
- Document network architecture and routing.
- Test connectivity during deployments and changes.



SENIOR ENGINEER TIP

Most VPC issues are routing problems. Follow the path, don't guess.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #6

VERIQTA

IAM ACCESS DENIED



PRODUCTION SCENARIO

Your application or user suddenly can't access AWS resources. Requests fail with AccessDenied errors. This can break deployments, data access, and critical workflows.



★ You try to access an AWS resource, but get **AccessDenied**. It's almost always a **permissions** or **policy** issue.

1. IAM POLICIES



- Attached to Users, Groups or Roles.
- Define what actions are allowed or denied.
- Identity-based policies.
- Evaluated in the account where the identity exists.



2. RESOURCE POLICIES



- Attached directly to resources (S3, SQS, SNS, KMS, etc.).
- Define who can access the resource.
- Resource-based policies.
- Can allow cross-account access.



3. ASSUMEROLE



- Allows an identity to assume a role and get temporary permissions.
- Uses STS (Security Token Service).
- Requires trust relationship in the role.



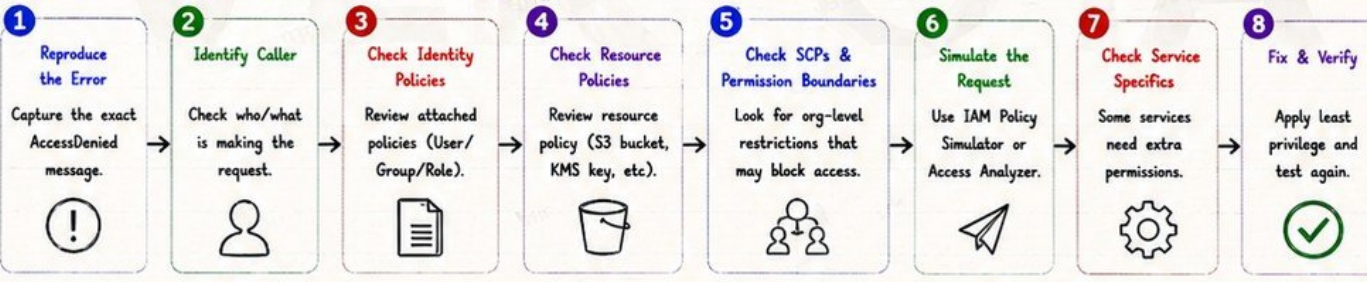
4. PERMISSION TROUBLESHOOTING



- Understand the request context.
- Identify which policy is blocking.
- Use AWS tools and logs.
- Fix the root cause, not just the symptom.



IAM PERMISSION INVESTIGATION WORKFLOW



EXAMPLE ACCESSDENIED ERROR

✖ AccessDenied
User: arn:aws:sts::123456789012:assumed-role/DeployRole/CodeDeploy is not authorized to perform: s3:PutObject on resource: arn:aws:s3::my-app-bucket/app.zip because no identity-based policy allows the s3:PutObject action

COMMON ROOT CAUSES

- Missing or incorrect IAM policy
- Resource policy explicitly denies access
- SCP (Service Control Policy) blocking the action
- Permission boundary restricting the action
- AssumeRole trust relationship misconfigured
- Session expired or invalid credentials
- Wrong region or account
- Typos in resource ARNs or actions
- MFA required but not used

Golden Rule:
Explicit Deny
always wins
over Allow.

POLICY EVALUATION LOGIC



USEFUL AWS CLI COMMANDS

```

# Get user/group/role policies
aws iam list-attached-user-policies \
  --user-name <username>

# Get role trust relationship
aws iam get-role --role-name <role-name> \
  --query 'Role.AssumeRolePolicyDocument'

# Simulate policy
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:role/DeployRole \
  --action-names s3:PutObject \
  --resource-arns arn:aws:s3::my-app-bucket/app.zip

# Check resource policy (S3 example)
aws s3api get-bucket-policy --bucket my-app-bucket

# Get caller identity
aws sts get-caller-identity

# Decode authorization message from error
aws sts decode-authorization-message \
  --encoded-message <encodedMessage>
  
```

BEST PRACTICES



Use least privilege principle



Review and clean up unused policies



Use meaningful names and tags



Use roles for applications, not users



Monitor Access with CloudTrail & CloudWatch



Test changes in lower environments first



SENIOR ENGINEER TIP

Always think in context:
Who is calling, What action,
On what resource, and
Which policies apply.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

7

INCIDENT #7

EBS VOLUME FULL

VERIQTA



- ★ Your EC2 instance or application becomes unstable or unresponsive. After investigation, you discover the EBS volume is **100% full**.

PRODUCTION SCENARIO

Logs, temporary files, or database growth consume all disk space on an EBS volume. Applications start failing, services crash, and users experience errors.



SYMPTOMS

- ✓ Applications return 500/503 errors
- ✓ Services crash or stop responding
- ✓ New files cannot be created
- ✓ Database failures
- ✓ CloudWatch alerts for disk usage
- ✓ OS reports "No space left on device"

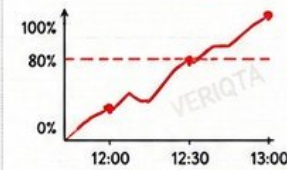


STORAGE MONITORING

CLOUDWATCH METRIC

- VolumeId: vol-xxxxxxx
- Metric: VolumeUsedPercent
- Alarm: > 80% or 90%
- Period: 5 minutes
- Statistic: Average
- Namespace: AWS/EBS

ALARM EXAMPLE



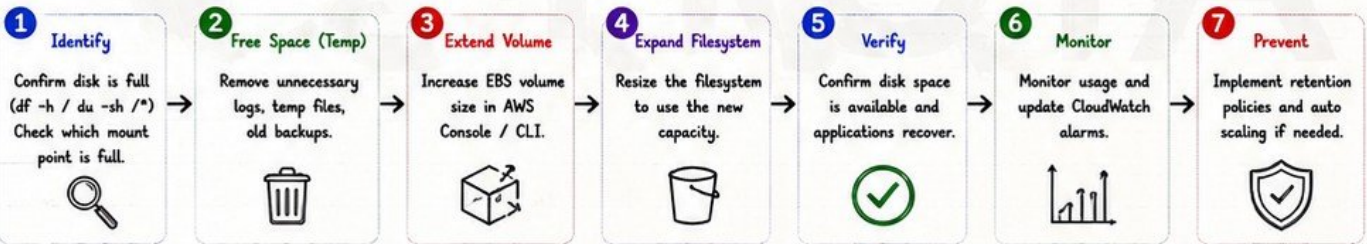
ALERT: VolumeUsedPercent > 90%

IMMEDIATE IMPACT

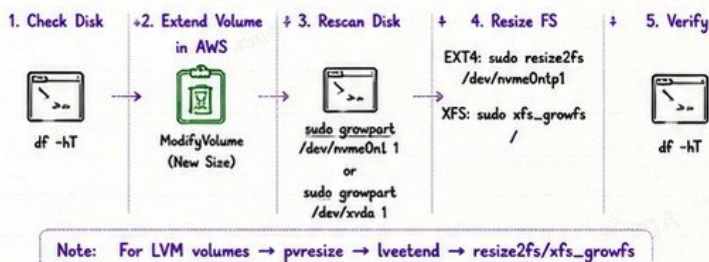
- ✗ Applications down or degraded
- ✗ Payments / Orders fail
- ✗ Data ingestion stops
- ✗ Services restart in a loop
- ✗ Customer experience severely affected
- ✗ SLA breach and revenue impact



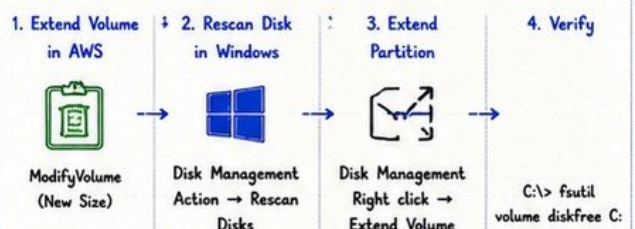
RECOVERY WORKFLOW



FILESYSTEM EXPANSION (LINUX)



FILESYSTEM EXPANSION (WINDOWS)



USEFUL CLI COMMANDS

```
# Check disk usage
df -h

# Find large directories
sudo du -sh /* 2>/dev/null | sort -rh | head -10

# Check inode usage
df -i

# Describe EBS volumes
aws ec2 describe-volumes --volume-ids vol-xxxxxxx

# Modify EBS volume size
aws ec2 modify-volume --volume-id vol-xxxxxxx --size 100

# Check volume modifications
aws ec2 describe-volumes-modifications --volume-ids vol-xxxxxxx
```

COMMON ROOT CAUSES



- ✗ Unbounded application logs
- ✗ No log rotation or retention policy
- ✗ Database growth without monitoring
- ✗ Old snapshots not cleaned up
- ✗ Temporary files not cleared
- ✗ Docker images / containers consuming space
- ✗ Lack of disk usage monitoring
- ✗ Wrong assumptions about storage needs

BEST PRACTICES



- ✓ Enable CloudWatch disk usage monitoring
- ✓ Set alarms at 70% / 80% usage
- ✓ Implement log rotation and retention
- ✓ Use Lifecycle policies for old data
- ✓ Regular cleanup of temp & unused files
- ✓ Right-size volumes and review monthly
- ✓ Use Auto Scaling & stateless design
- ✓ Document runbooks for disk issues



SENIOR ENGINEER TIP:

Disk full issues are preventable. Monitor, set alarms early, and automate cleanup. React fast, but fix the root cause.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #8

VERIQTA

AUTO SCALING GROUP NOT LAUNCHING INSTANCES

aws

PRODUCTION SCENARIO

Your Auto Scaling Group (ASG) is not launching instances even though demand is high or desired capacity > current capacity. Applications become slow or unavailable.

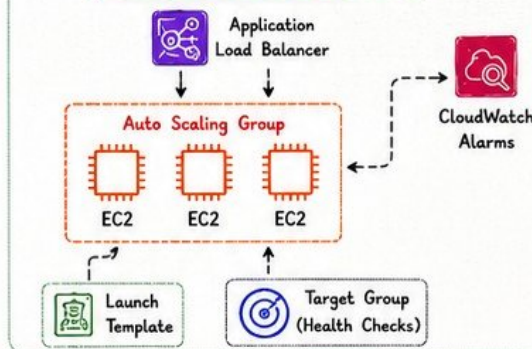
- ★ The ASG should launch new instances, but desired capacity is **not met**. This can lead to **performance issues** or **full outages**.

COMMON SYMPTOMS

- ✓ Desired capacity > Current capacity
- ✓ No new instances launching
- ✓ Scaling activities show "Failed"
- ✓ High CPU / request count
- ✓ Users experience slowness or errors
- ✓ CloudWatch alarms triggered but capacity doesn't increase



HIGH-LEVEL ARCHITECTURE

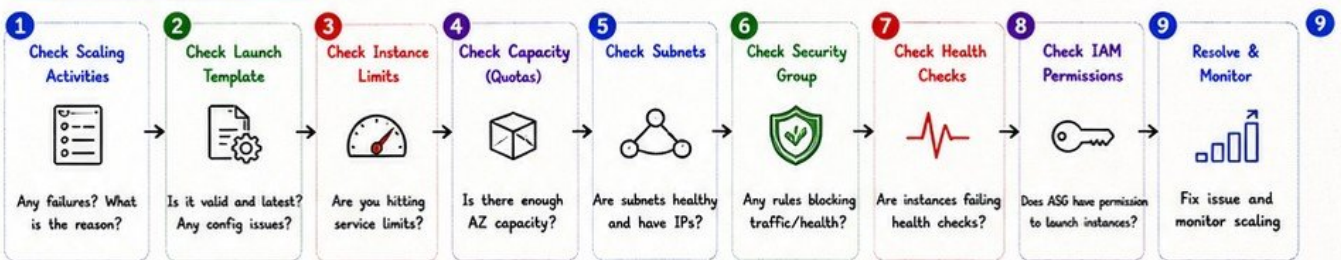


IMMEDIATE IMPACT

- ✗ Applications slow or unavailable
- ✗ Inability to handle traffic spikes
- ✗ Failed scaling activities
- ✗ SLA breaches
- ✗ Customer dissatisfaction



INVESTIGATION WORKFLOW



1. LAUNCH TEMPLATES CHECKLIST

- ✓ Template exists and is the latest version
- ✓ AMI is valid and available
- ✓ Instance type is supported in your AZs
- ✓ Key pair, IAM role, SGs correctly configured
- ✓ UserData / scripts are not failing
- ✓ Block device mappings valid (EBS limits)



2. SCALING POLICIES

- Target Tracking**
 - Maintains a target metric (e.g., CPU 60%)
 - Adjusts capacity automatically
- Step Scaling**
 - Adds/removes instances when thresholds are crossed
 - Define upper & lower thresholds
- Scheduled Scaling**
 - Changes capacity based on time
 - Great for predictable workloads

3. CAPACITY INVESTIGATION

- ✗ **Check Service Limits**
aws service-quotas list-service-quotas --service-code ec2
- ✗ **Check On-Demand Limits**
Are you hitting vCPU or instance limits?
- ✗ **Check AZ Capacity**
Not enough capacity in one AZ → launch fails.
- ✗ **Use Multiple AZs**
For high availability and better capacity availability.



USEFUL AWS CLI COMMANDS

```
# Describe scaling activities
aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name <asg-name>
# Describe Auto Scaling Groups
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names <asg-name>
# Describe launch templates
aws ec2 describe-launch-templates \
  --launch-template-names <lt-name>
# Check service quotas
aws service-quotas list-service-quotas --service-code ec2
```

COMMON ROOT CAUSES

- ✗ Incorrect or outdated Launch Template
- ✗ Instance type not available in AZ
- ✗ Subnet without available IP addresses
- ✗ ASG hitting instance or vCPU limits
- ✗ Security Group blocking health checks
- ✗ Failed UserData or bootstrap scripts
- ✗ IAM role missing permissions
- ✗ Quota limits reached



BEST PRACTICES

- ✓ Use latest Launch Template versions
- ✓ Distribute across multiple AZs
- ✓ Monitor scaling activities & CloudWatch alarms
- ✓ Set appropriate min/max/desired capacity
- ✓ Test scaling regularly
- ✓ Keep service quotas high enough for growth
- ✓ Use Target Tracking for dynamic workloads



SENIOR ENGINEER TIP:

Always start by checking **scaling activities**. The error message will usually point you to the **root cause**. Fix the first blocking issue, then re-check scaling.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #9

ELASTIC LOAD BALANCER MARKS TARGETS UNHEALTHY



PRODUCTION SCENARIO

Users report 503 errors.
The load balancer is reachable, but all or some targets in the Target Group are unhealthy.
No traffic is routed to them.



★ ELB is running, but your targets are marked **UNHEALTHY**.
This causes **5xx errors**, **traffic routing issues**, and **outages**.

HEALTH CHECKS

- ✓ ELB sends periodic requests to targets to verify they are healthy.
- ✓ Defined at the Target Group level.
- ✓ Supports HTTP, HTTPS, TCP, and gRPC.
- ✓ Based on successful response code, timeouts, and thresholds.

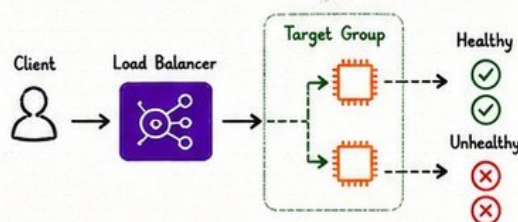


TARGET GROUPS

- ✓ A logical group of targets (instances, IPs, Lambda) behind the load balancer.
- ✓ ELB routes traffic only to healthy targets.
- ✓ Each target has its own health status.
- ✓ Unhealthy targets are automatically removed from rotation.



HOW IT WORKS



COMMON CAUSES OF TARGETS UNHEALTHY

Health Check Misconfiguration



Wrong path, port, protocol, or expected response code.

Application Not Responding



App down, crash, or long response time.

Security Group Blocking



ELB cannot reach target on the health check port.

NACL Blocking



Network ACL rules blocking return traffic.

Instance Issues



High CPU, OOM, disk full, or system unreachable.

Target Unregistered



Instance removed or failed to register with Target Group.

Timeouts / Latency



Health check timeout too low for slow apps.

Incorrect Listener Rule



Traffic routed to wrong target group or port.

INVESTIGATION WORKFLOW



1 Check Target Health



`aws elbv2 describe-target-health --target-group-arn <tg-arn>`

2 Review Health Check Settings



Check path, port, protocol, thresholds, and expected codes.

3 Test From ELB To Target



Use VPC reachability or SSM to test connectivity.

4 Check Security Groups & NACLs



Ensure ELB SG can reach target on health check port.

5 Check Application Logs



Look for errors, crashes, or slow responses.

6 Verify Target Registration



Ensure instances are registered and in the correct AZ.

7 Fix & Re-check Health



Apply fix and wait for targets to return to healthy.

USEFUL AWS CLI COMMANDS



```
# List target health
aws elbv2 describe-target-health --target-group-arn <tg-arn>

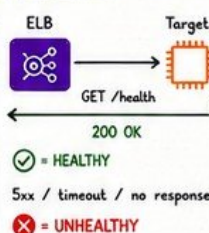
# Describe target group
aws elbv2 describe-target-groups --target-group-arn <tg-arn>

# Describe listeners
aws elbv2 describe-listeners --load-balancer-arn <lb-arn>

# Test from instance (Linux)
curl -v http://<target-ip>:<port>/<health-check-path>
```

EXAMPLE HEALTH CHECK (HTTP)

Protocol: HTTP
Path: /health
Port: 8080
Matcher: 200-399
Interval: 30s
Timeout: 5s
Healthy threshold: 3
Unhealthy threshold: 3



BEST PRACTICES



- ✓ Configure health checks specific to your application.
- ✓ Use dedicated lightweight health endpoints.
- ✓ Set appropriate timeouts and thresholds.
- ✓ Keep security groups simple and least privilege.
- ✓ Monitor target health with CloudWatch and alarms.
- ✓ Test after every change.



SENIOR ENGINEER TIP:

Unhealthy targets mean ELB sees a problem. Always validate from the ELB's perspective, not just from inside the instance.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #10

VERIQTA

RDS DATABASE SUDDENLY UNREACHABLE



PRODUCTION SCENARIO

Applications start failing with database connection errors.
Users see 500 errors.
The database endpoint is not reachable.



★ Your application can't connect to the RDS database.
This can be caused by **connectivity**, **failover**, **security**, or **DNS** issues.

COMMON SYMPTOMS

- ✓ Application connection timeout
- ✓ Can't connect to endpoint/port
- ✓ High latency or intermittent failures
- ✓ RDS status shows "Available" but connections fail
- ✓ Transactions fail or rollbacks increase



DATABASE CONNECTIVITY

- ✓ Clients connect to RDS using the endpoint and port.
- ✓ Traffic flows through VPC, Security Groups, and NACLs.
- ✓ RDS responds on the DB port (e.g., 3306 for MySQL, 5432 for PostgreSQL).
- ✓ If any layer blocks traffic, connection fails.



FAILOVER SCENARIO

- 1 RDS detects a failure (instance/AZ down).
- 2 Automatic Multi-AZ failover is triggered.
- 3 DNS endpoint is updated to point to the new primary.
- 4 New primary becomes available and application reconnects.

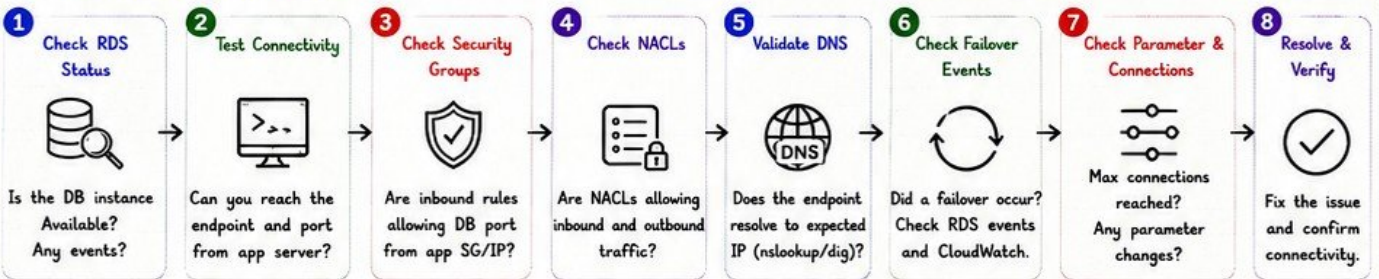


IMMEDIATE IMPACT

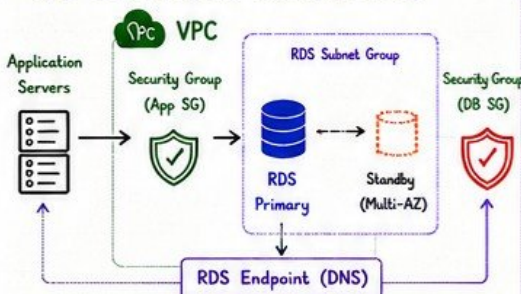
- ✗ Applications unable to connect
- ✗ User-facing errors (500)
- ✗ Transactions fail
- ✗ Connection pool exhausted
- ✗ Time-outs increase
- ✗ SLA and revenue at risk



INVESTIGATION WORKFLOW



RDS CONNECTIVITY ARCHITECTURE



1. App connects to RDS Endpoint (DNS name)
2. DNS resolves to Primary IP
3. Traffic allowed by SGs and NACLs
4. RDS responds on DB port
5. Failover updates DNS to new Primary

USEFUL AWS CLI COMMANDS



```

# Describe DB instances
aws rds describe-db-instances \
  --query 'DBInstances[*].[DBInstanceIdentifier,DB InstanceStatus,Endpoint.Address,Engine]' \
  --output table

# Describe recent events
aws rds describe-events \
  --source-identifier mydb \
  --max-items 20 --output table

# Test connectivity from EC2
nc -vz mydb.cluster-xyz.region.rds.amazonaws.com 3306

# Check DNS resolution
nslookup mydb.cluster-xyz.region.rds.amazonaws.com

# Check SG rules
aws ec2 describe-security-groups \
  --group-ids sg-xxxxxx
  
```

COMMON ROOT CAUSES

- ✗ Security Group blocking DB port
- ✗ NACL rules blocking traffic
- ✗ Wrong DB endpoint or hardcoded IP
- ✗ DNS caching / stale resolution
- ✗ Failover in progress (old primary down)
- ✗ DB instance stopped or rebooting
- ✗ Parameter change requiring reboot
- ✗ Storage full causing instance issues
- ✗ Max connections reached



SECURITY GROUP CHECKLIST

- ✓ Inbound rule allows DB port (3306 / 5432 / 1521 etc.)
- ✓ Source is correct (App SG or CIDR)
- ✓ No conflicting deny rules
- ✓ Outbound rules allow responses



DNS INVESTIGATION

- 1 Use nslookup or dig
- 2 Verify it resolves to valid IP
- 3 Check TTL (low TTL = faster failover)
- 4 Flush DNS cache if needed



BEST PRACTICES

- ✓ Use Multi-AZ for high availability
- ✓ Use RDS Proxy for connection management
- ✓ Keep SGs least-privilege
- ✓ Monitor RDS health and events
- ✓ Test failover regularly



SENIOR ENGINEER TIP

When the database is unreachable, think in **network** → DNS. Validate layer: **dig** **hijack** before jumping to conclusions.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

INCIDENT #11

VERIQTA

CLOUDWATCH ALARM NEVER TRIGGERS



PRODUCTION SCENARIO

CPU is pegged at 95%, applications are slow, but the CloudWatch alarm you created never goes off. You get paged by users, not by CloudWatch.



- ★ You created a CloudWatch Alarm, but it never triggers, even when the problem clearly exists. The issue is usually in **metrics**, **thresholds**, or **configuration**.

COMMON SYMPTOMS

- ✓ Alarm state stays OK
- ✓ No notifications sent
- ✓ Metric value is high but alarm does not trigger
- ✓ Alarm shows "INSUFFICIENT_DATA"
- ✓ Users report issue before alarm
- ✓ Dashboard shows no metric data



KEY CONCEPTS



METRICS
CloudWatch collects data from AWS services, applications, and custom sources.



THRESHOLDS
Alarm triggers when metric value breaches the defined threshold for a period.

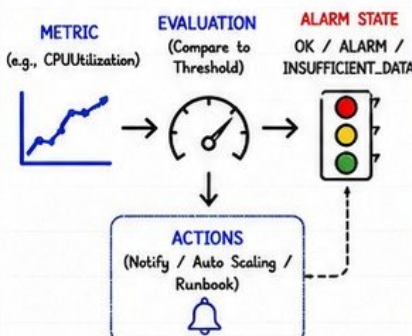


MISSING DATA
No data points = alarm may not change state depending on settings.



ALARM ACTIONS
Notifications, Auto Scaling, Lambda, SNS, SSM, etc.

HOW CLOUDWATCH ALARMS WORK

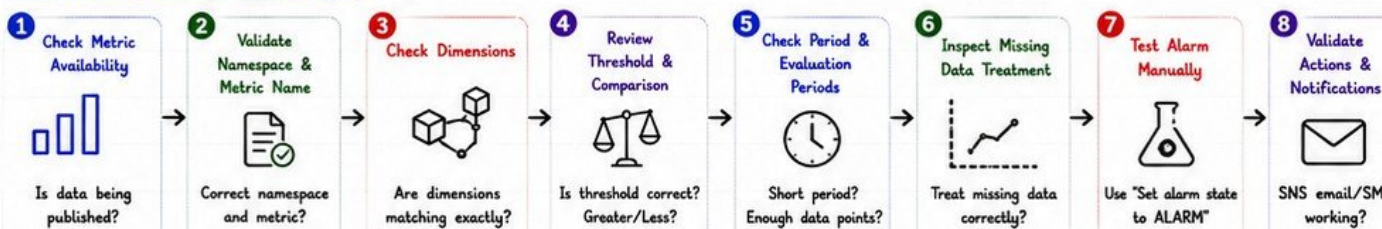


IMMEDIATE IMPACT

- ✗ Incidents not detected early
- ✗ Outages become larger
- ✗ Alerts come too late (users first)
- ✗ SLA and trust impact
- ✗ Missed Auto Scaling
- ✗ Operations and cost impact



ALARM DEBUGGING WORKFLOW



METRIC EXAMPLES

- CPUUtilization (AWS/EC2)
- TargetResponseTime (AWS/ApplicationELB)
- 5xx (AWS/ApplicationELB)
- DatabaseConnections (AWS/RDS)
- Custom Business Metric (PutMetricData)

MISSING DATA TREATMENT OPTIONS

OPTION	BEHAVIOR
IGNORE	Keep last state. Missing data has no impact.
BREACHING	Treat missing data as ALARM. Good for critical metrics.
NOT BREACHING	Treat missing data as OK. May hide real issues.
MISSING	Set state to INSUFFICIENT_DATA when no data.

REASONS ALARM NEVER TRIGGERS

- ✗ No metric data is being published
- ✗ Wrong namespace or metric name
- ✗ Dimensions do not match actual resources
- ✗ Threshold is too high or unrealistic
- ✗ Wrong comparison operator (>, <, >=, <=)
- ✗ Period/Evaluation periods misconfigured
- ✗ Missing data treated as NOT BREACHING
- ✗ Insufficient data points for evaluation
- ✗ Alarm in wrong region or account



USEFUL AWS CLI COMMANDS

```

# List alarms
aws cloudwatch describe-alarms

# Get alarm details
aws cloudwatch describe-alarms \
  --alarm-names <alarm-name>

# Get metric data
aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --statistics Average \
  --start-time 2024-07-16T10:00:00Z \
  --end-time 2024-07-16T11:00:00Z \
  --period 60 --dimensions Name=InstanceId,Value=i-1234567890abcdef0

# Test alarm state
aws cloudwatch set-alarm-state \
  --alarm-name <alarm-name> \
  --state-value ALARM \
  --state-reason "Testing alarm"
  
```

BEST PRACTICES

- ✓ Verify metrics before creating alarms
- ✓ Use meaningful thresholds
- ✓ Choose correct period & evaluation periods
- ✓ Use BREACHING for critical alarms
- ✓ Test alarms periodically
- ✓ Monitor alarm state changes
- ✓ Set up proper notifications & runbooks
- ✓ Document alarms and owners



SENIOR ENGINEER TIP

Always start with the data. If CloudWatch doesn't see it, it can't alert on it. Trust the metric, then trust the alarm.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

12

INCIDENT #12

VERIQTA

APPLICATION CANNOT ACCESS



S3



PRODUCTION SCENARIO

Your application tries to read or write to an S3 bucket, but gets Access Denied or times out. Business operations depending on S3 fail.



★ Your application or users cannot access objects in S3.

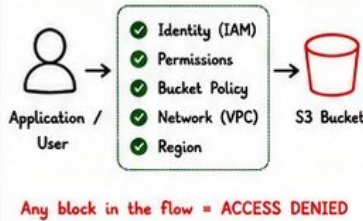
The issue is usually in **permissions**, **policies**, **network**, or **region**.

COMMON SYMPTOMS

- ✓ Access Denied (403) errors
- ✓ NoSuchBucket or AccessDenied
- ✓ Timeout when accessing S3
- ✓ Uploads or downloads fail
- ✓ Works from console, not from app
- ✓ Only fails in one environment



BUCKET ACCESS FLOW



COMMON CAUSE CATEGORIES

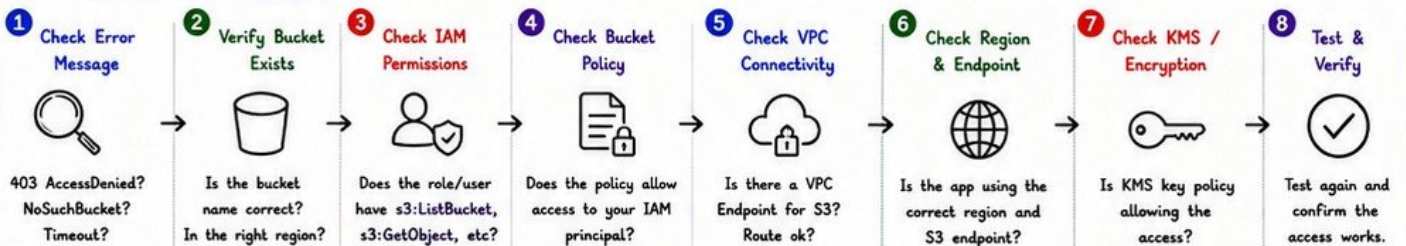
- Bucket Policies**
Deny or missing allow
- IAM Permissions**
Missing or insufficient permissions
- Network Issues**
No route / VPC endpoint / firewall
- Region Issues**
Wrong bucket region or endpoint

IMMEDIATE IMPACT

- ✗ Application features fail
- ✗ Users experience errors
- ✗ Data uploads/downloads stop
- ✗ Reports and backups fail
- ✗ SLA and ETL jobs impacted
- ✗ Revenue and trust affected



INVESTIGATION WORKFLOW



BUCKET POLICY EXAMPLE

```

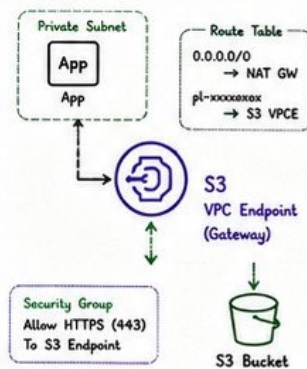
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:role/AppRole"
      },
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": [
        "arn:aws:s3::my-app-bucket",
        "arn:aws:s3::my-app-bucket/*"
      ]
    }
  ]
}
  
```

IAM POLICY EXAMPLE

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3::my-app-bucket"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3::my-app-bucket/*"
    }
  ]
}
  
```

VPC ENDPOINT FOR S3



REGION ISSUES

- ✗ Bucket exists in us-east-1 but app uses eu-west-1 endpoint.
- ✗ Using wrong endpoint (e.g., s3.amazonaws.com instead of s3.eu-west-1.amazonaws.com)
- ✗ Cross-region access without proper replication / config.



QUICK CHECKLIST

- ✓ Bucket name and region are correct
- ✓ IAM role has required S3 permissions
- ✓ Bucket policy allows the principal
- ✓ No explicit Deny blocking access
- ✓ VPC endpoint for S3 exists (if private)
- ✓ KMS key policy allows the principal
- ✓ Test from same environment as app



USEFUL AWS CLI COMMANDS

```

# Check who you are
aws sts get-caller-identity
# Check bucket location
aws s3api get-bucket-location --bucket my-app-bucket
# Check bucket policy
aws s3api get-bucket-policy --bucket my-app-bucket
# List objects
aws s3 ls s3://my-app-bucket --region us-east-1
# Test access
aws s3 cp test.txt s3://my-app-bucket/ --region us-east-1
  
```

BEST PRACTICES

- ✓ Use least privilege IAM policies
- ✓ Avoid * in bucket policies
- ✓ Use VPC endpoints for private access
- ✓ Keep bucket and app in same region
- ✓ Enable CloudTrail for auditing
- ✓ Monitor access errors in CloudWatch
- ✓ Test access after any policy change



SENIOR ENGINEER TIP:

S3 access issues are 90% permissions or policy related. Always check IAM → Bucket Policy → Network → Region in that order. Use CloudTrail to see the exact reason for denied access.



INCIDENT #13

VERIQTA

HIGH EC2
CPU UTILIZATION

PRODUCTION SCENARIO

Users report slow application response or timeouts.
CPU on EC2 is high for a long period, impacting availability and user experience.



- ★ Your EC2 instance CPU is constantly high (80-100%).
The issue is usually in **performance**, **workload**, or **configuration**.

COMMON SYMPTOMS

- ✓ CPU Utilization > 80% for extended time
- ✓ High load average (load > # vCPU)
- ✓ Application slow or unresponsive
- ✓ Request queues growing
- ✓ Increased latency and errors
- ✓ Auto Scaling scaling out frequently



KEY CONCEPTS



CloudWatch Metrics
Monitor CPU Utilization, Load Average, NetworkIn, DiskIO.



Performance Bottlenecks
CPU, memory, disk, network, or application issues.



Scaling Decisions
Scale up (bigger instances) or scale out (more instances).



Right-sizing
Match instance size to workload.

COMMON CAUSES



High traffic / unexpected load



Inefficient code or heavy processes



Background jobs / cron tasks



Memory pressure causing swapping



Disk or network bottlenecks



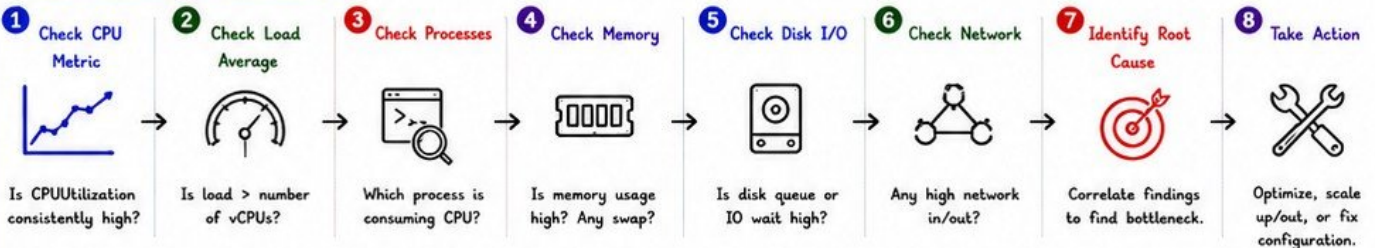
Instance type too small (under-provisioned)

IMMEDIATE IMPACT

- ✗ Application becomes slow
- ✗ Users experience timeouts
- ✗ Services may become unavailable
- ✗ Auto Scaling costs increase
- ✗ SLA and revenue impacted
- ✗ Related systems get affected



INVESTIGATION WORKFLOW



CLOUDWATCH METRICS TO MONITOR



CPU Utilization (%) Primary indicator of CPU load



Load Average (1m/5m/15m) Compare with vCPU count



NetworkIn / NetworkOut High network can cause CPU spikes



DiskReadOps / DiskWriteOps High I/O increases CPU usage



StatusCheckFailed (System) Detect underlying system issues

IDENTIFY TOP CPU CONSUMERS (ON INSTANCE)

```
$ top -o %CPU
```

PID	USER	%CPU	COMMAND
2345	www-data	85.1	node server.js
1243	root	45.3	python heavy_task.py
5678	mysql	25.7	mysqld
9101	root	12.4	amazon-cloudwatch-agent

Other useful commands:

- ✓ \$ ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head
- ✓ \$ mpstat -P ALL 1 5
- ✓ \$ iostat -xz 1 5
- ✓ \$ sar -u 1 5

SCALING DECISIONS



SCALE UP (Vertical)

- ✗ Move to bigger instance type
- ✗ Use when single instance is the bottleneck
- ✗ Example: t3.medium → t3.large



SCALE OUT (Horizontal)

- ✗ Add more instances behind Load Balancer
- ✗ Use Auto Scaling Group
- ✗ Example: 2 → 4 → 8 instances

QUICK CHECKLIST

- ✓ Confirm high CPU in CloudWatch
- ✓ Check load average vs vCPUs
- ✓ Identify top CPU consuming process
- ✓ Check memory, disk, network
- ✓ Confirm no runaway or cron job
- ✓ Decide: optimize, scale up or scale out



USEFUL AWS CLI COMMANDS



```
# Get CPU metrics for instance
aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=i-1234567890abcdef0 \
  --start-time 2024-07-16T10:00:00Z \
  --end-time 2024-07-16T11:00:00Z \
  --period 60 --statistics Average

# Describe instance type
aws ec2 describe-instance-types \
  --instance-types t3.medium
```

BEST PRACTICES

- ✓ Right-size your instances
- ✓ Use Auto Scaling for variable load
- ✓ Optimize application and queries
- ✓ Offload heavy tasks to background workers
- ✓ Use caching (Redis / Memcached)
- ✓ Monitor with CloudWatch dashboards
- ✓ Set CPU alarms and act early



SENIOR ENGINEER TIP

High CPU is a symptom, not the disease.
Always find the bottleneck → fix at the root, then scale smart.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

NAT GATEWAY FAILURE



PRODUCTION SCENARIO

Instances in private subnets cannot access the internet or AWS services (updates, external APIs, package installs). The NAT Gateway is down or misconfigured.



- ★ Private subnet resources need a NAT Gateway to reach the internet.
If the NAT Gateway fails, outbound traffic **breaks**.

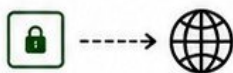
COMMON SYMPTOMS

- ✓ Instances cannot access the internet
- ✓ Cannot download updates / packages
- ✓ apt-get update / yum update fails
- ✓ Cannot reach external APIs
- ✓ AWS CLI calls fail from private subnet
- ✓ No outbound connectivity
- ✓ Health checks show NAT down



WHAT IS A NAT GATEWAY?

- ✓ A managed service that allows instances in private subnets to connect to the internet or other AWS services.
- ✓ It is placed in a Public Subnet and has an Elastic IP.



COMMON CAUSES

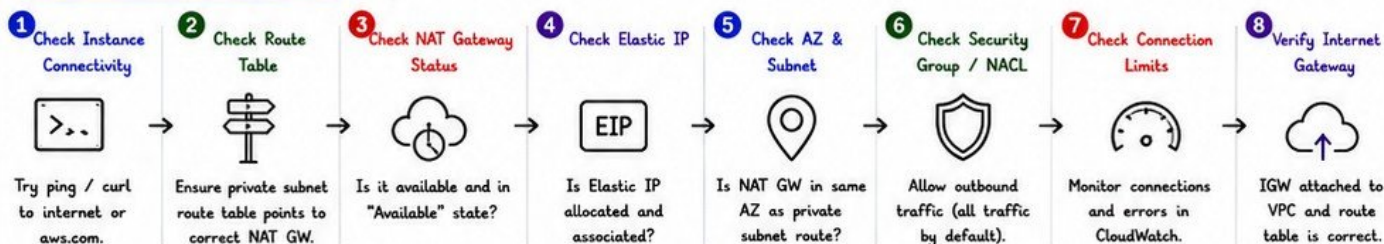
- ✗ NAT Gateway is deleted or stopped
- ✗ Elastic IP is released
- ✗ Subnet route table points to wrong NAT
- ✗ NAT Gateway in one AZ down (single AZ setup)
- ✗ Quota / connection tracking exhausted
- ✗ Misconfigured security group (or NACL)

IMMEDIATE IMPACT

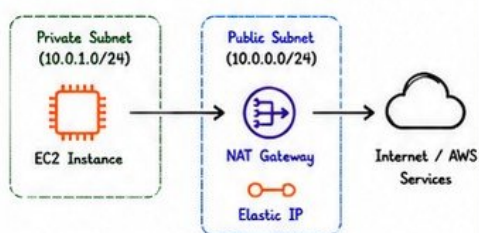
- ✗ No internet access for private subnet resources
- ✗ Application updates fail
- ✗ External dependencies fail
- ✗ CI/CD jobs in private subnet fail
- ✗ Monitoring / logging may stop
- ✗ Business operations impacted



INVESTIGATION WORKFLOW



NETWORK ARCHITECTURE (NORMAL)



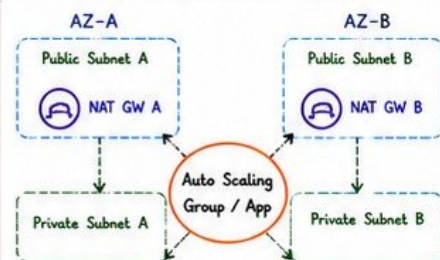
1. Instances route 0.0.0.0/0 to NAT Gateway
2. NAT Gateway routes traffic to Internet via IGW
3. Responses return back through NAT Gateway

CHECK ROUTE TABLE EXAMPLE

Private Subnet Route Table	
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-0abc123def456

Should point to NAT Gateway

HIGH AVAILABILITY DESIGN



- ✓ Deploy NAT Gateway in each AZ
- ✓ Route tables use local AZ NAT Gateway

QUICK CHECKLIST

- ✓ Private subnet route table → NAT GW
- ✓ NAT Gateway status = Available
- ✓ Elastic IP allocated & associated
- ✓ NAT GW in same AZ (recommended)
- ✓ Security Group allows outbound
- ✓ IGW attached to VPC



USEFUL AWS CLI COMMANDS

```
# Describe NAT Gateways
aws ec2 describe-nat-gateways
# Check NAT Gateway status
aws ec2 describe-nat-gateways \
  --nat-gateway-ids nat-0abc123def456
# Describe Route Tables
aws ec2 describe-route-tables \
  --filters "Name=vpc-id,Values=vpc-123"
# Check Elastic IP
aws ec2 describe-addresses
# Check Connection Errors in CloudWatch
aws cloudwatch get-metric-statistics \
  --namespace AWS/NATGateway
```

BEST PRACTICES

- ✓ Use NAT Gateway in each AZ for HA
- ✓ Monitor NAT Gateway metrics
- ✓ Set CloudWatch Alarms for Errors
- ✓ Right-size and monitor connection usage
- ✓ Use VPC Endpoints for AWS services
- ✓ Test internet connectivity during deploys



SENIOR ENGINEER TIP

If your application in private subnets can't reach the internet, always check: Route Table → NAT Gateway → Elastic IP → IGW.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

UNEXPECTED AWS COST SPIKE



★ Your AWS bill suddenly spikes without warning.
The issue is usually in **usage**, **configuration**, or **missing controls**.

PRODUCTION SCENARIO

You open your bill and see a huge increase compared to yesterday.
Uncontrolled resources or hidden usage is consuming your budget.
Action needed immediately!



COMMON SYMPTOMS

- Bill increased dramatically
- No recent architecture changes
- Unusual services or regions active
- EC2 running 24/7 by mistake
- High data transfer or NAT costs
- S3 storage growing fast
- Unattached EBS / IP addresses
- No cost alerts configured



KEY CONCEPTS



Cost Explorer
Analyze historical and daily spend.



CloudWatch Metrics
Monitor usage in near real-time.



Budgets & Alerts
Set alerts before cost becomes critical.



Tagging Strategy
Tag everything to track cost by team / project.

COMMON PRODUCTION CAUSES



Unstopped EC2 / RDS instances



Auto Scaling gone out of control



Large data transfer (internet / NAT)



S3 logging / backups growing



Unbounded snapshots / AMIs



Expensive services left running (Elasticache, OpenSearch, etc.)



Cross-AZ or cross-region transfers



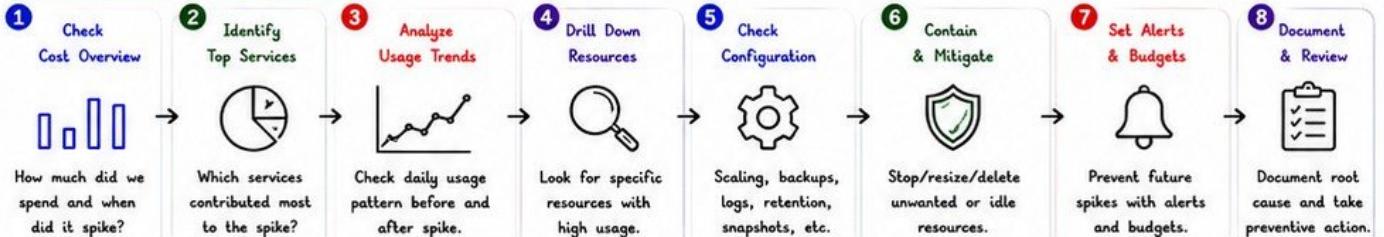
No budgets or alerts in place

IMMEDIATE IMPACT

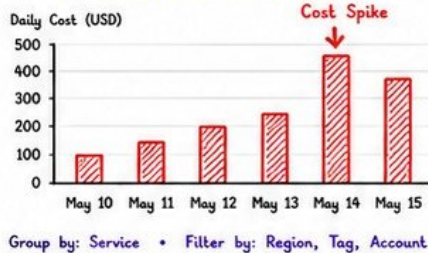
- High AWS bill and over-budget
- Services can be throttled or stopped
- Savings Plans / Credits exhausted
- Business and team unhappy
- Risk of account suspension
- Reduced trust in cloud operations



INVESTIGATION WORKFLOW



COST EXPLORER EXAMPLE



CLOUDWATCH METRICS TO WATCH

- EstimatedCharges (AWS/Billing)
- EC2 CPUUtilization
- RDS DatabaseConnections
- S3 BucketSizeBytes
- NetworkOut / BytesOut
- NATGateway BytesOut
- Lambda Invocations



COMMON COST DRIVERS

- EC2 instances left running (24/7)
- Auto Scaling group misconfigured
- Large data transfer (internet, NAT, cross-region)
- S3 storage and request charges
- Unnecessary snapshots / AMIs / volumes
- RDS in Multi-AZ when not needed
- High CloudWatch Logs ingestion
- Over-provisioned resources



COST OPTIMIZATION WORKFLOW



USEFUL AWS CLI COMMANDS

```
# Get daily cost and usage
aws ce get-cost-and-usage \
  --time-period Start=2024-05-01,End=2024-05-15 \
  --granularity DAILY \
  --metrics BlendedCost

# Top 10 services by cost
aws ce get-cost-and-usage \
  --time-period Start=2024-05-01,End=2024-05-15 \
  --group-by Type=DIMENSION,Key=SERVICE \
  --metrics BlendedCost \
  --max-results 10

# Get cost for specific service
aws ce get-cost-and-usage \
  --time-period Start=2024-05-01,End=2024-05-15 \
  --filter '[{"Dimensions":{"Key":"SERVICE","Values":["AmazonEC2"]}}]' \
  --metrics BlendedCost
```

BEST PRACTICES

- Enable Budgets and cost alerts
- Tag all resources (owner, env, project)
- Use Auto Scaling with limits
- Stop resources when not needed
- Use Savings Plans / Reserved Instances
- Monitor CloudWatch and Cost Explorer
- Review costs weekly
- Automate cleanup with Lambda / scripts



SENIOR ENGINEER TIP

Cost spikes are a symptom, not the root cause.
Always automate monitoring, tagging, and cleanup.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

SENIOR AWS INCIDENT INVESTIGATION WORKFLOW

★ Senior engineers follow a structured, data-driven approach.
Purpose: find the **real root cause** and fix it permanently.

SENIOR MINDSET

- ✓ Stay calm. Don't panic.
- ✓ Verify before assuming.
- ✓ Use data, not opinions.
- ✓ Think in dependencies.
- ✓ Document everything.
- ✓ Fix the cause, not symptoms.



HOW SENIOR ENGINEERS APPROACH INCIDENTS

- 1 Understand the impact
- 2 Gather facts & data
- 3 Form a hypothesis
- 4 Test the hypothesis
- 5 Drill down to root cause
- 6 Implement the fix
- 7 Validate the fix
- 8 Document & improve



SENIOR INVESTIGATION PRINCIPLES

- ✓ Start broad, then narrow down
- ✓ Use the AWS Well-Architected framework
- ✓ Check the easy things first
- ✓ Correlate across services & time
- ✓ Look for recent changes
- ✓ Ask: What, When, Where, Who, Why, How?
- ✓ Communicate early & clearly
- ✓ Blameless, focus on learning



SENIOR TOOLKIT (AWS)

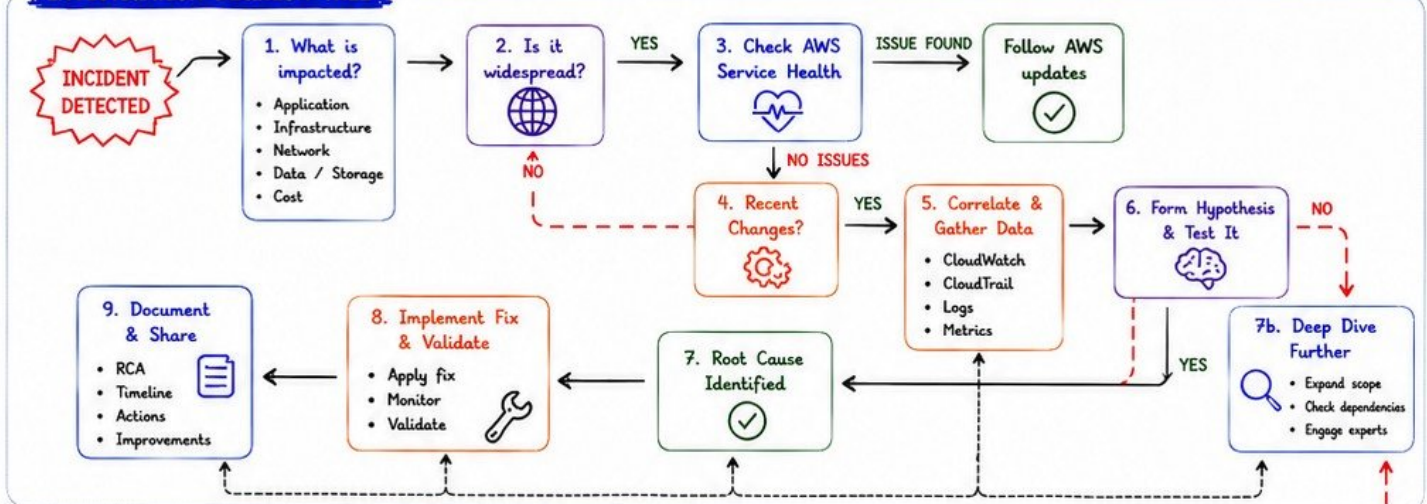
- CloudWatch (Metrics, Logs, Alarms)
- CloudTrail (API activity)
- AWS Config (Changes)
- X-Ray (Tracing)
- VPC Flow Logs
- Service Health Dashboard
- AWS Support
- Cost Explorer (Cost Incidents)
- Trusted Advisor

WHAT TO AVOID

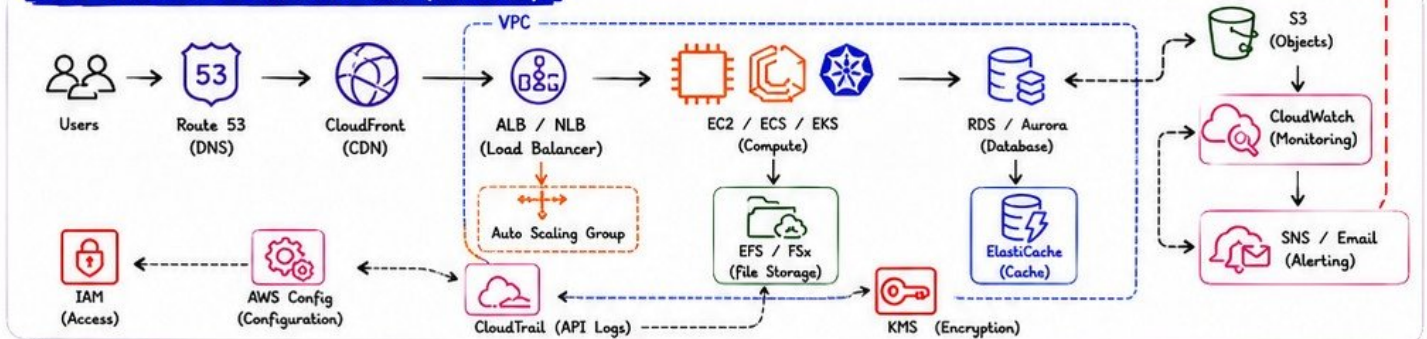
- ✗ Jumping to conclusions
- ✗ Rebooting / Restarting blindly
- ✗ Ignoring recent changes
- ✗ Siloed investigation
- ✗ Not validating the fix
- ✗ No documentation
- ✗ User blame
- ✗ Fixing symptoms only



INVESTIGATION DECISION TREE



AWS SERVICE DEPENDENCY MAP (EXAMPLE)



SENIOR TIP

Map the dependencies. When one service fails, others feel the impact. Follow the chain!



WELL-ARCHITECTED LENSES



REMEMBER

Data tells the story. Your job is to listen, connect the dots, fix the real problem, then make it better!



AWS PRODUCTION ARCHITECTURE

BEST PRACTICES



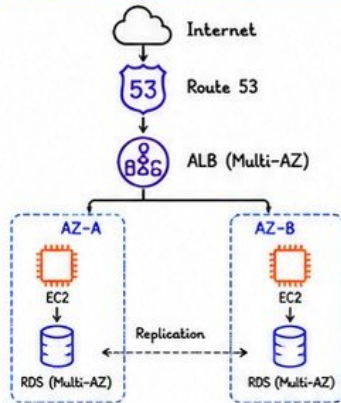
★ Design for **reliability**, **performance**, **security** and **cost-efficiency**.
Follow these principles for production-ready systems.

ARCHITECTURE PRINCIPLES

- ✓ Stop guessing capacity
- ✓ Plan for failure
- ✓ Design for scale
- ✓ Automate everything
- ✓ Monitor continuously
- ✓ Improve constantly



1 HIGHLY AVAILABLE ARCHITECTURE



- ✓ Use Multi-AZ for all critical components
- ✓ Spread resources across at least 2 AZs
- ✓ Remove single points of failure

2 MULTI-AZ (EVERYTHING THAT MATTERS)

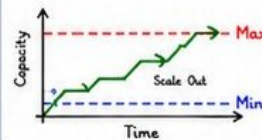
- ALB / NLB
Deploy across multiple AZs
- EC2 Auto Scaling Group
Distribute instances evenly
- RDS / Aurora
Use Multi-AZ for automatic failover
- ElastiCache / Redis
Enable Multi-AZ
- EFS / FSx
Use regional services
- NAT Gateway
Deploy one NAT GW per AZ

Multi-AZ ≠ Multi-Region.
AZ protects from AZ failures,
Region protects from region failures.

3 AUTO SCALING

EC2 Auto Scaling Group

- ✓ Scale out based on demand
- ✓ Use Target Tracking (CPU, RPS)
- ✓ Set Min, Desired, Max wisely
- ✓ Use Warm Pools for faster scale
- ✓ Health checks + Automatic replacement



4 MONITORING & OBSERVABILITY

- CloudWatch
Metrics, Alarms, Dashboards
- CloudTrail
API activity logging
- X-Ray
Distributed tracing
- AWS Config
Resource configuration
- Logs
Centralize with Logs Insights

You can't fix what you don't monitor!



SECURITY BEST PRACTICES

- IAM Least Privilege
MFA for all users
- Security Groups
Restrict inbound / outbound
- WAF
Protect your applications
- Secrets Manager
Store & rotate secrets
- KMS Encryption
Encrypt data at rest
- VPC Design
Private subnets for workloads

SECURITY CHECKLIST

- ✓ No public access to databases
- ✓ Use Private Subnets
- ✓ Encrypt data in transit (TLS) & at rest
- ✓ Patch & update regularly
- ✓ Enable GuardDuty, Security Hub, Inspector



BACKUP STRATEGY

- RDS Automated Backups
Enable & test restoration
- EBS Snapshots
Schedule regular snapshots
- S3 Versioning
Protect against accidental deletes
- AWS Backup
Centralized backup management
- Test Restores Regularly
Backups are useless if you can't restore

Follow the 3-2-1 rule:
3 copies, 2 media types, 1 offsite.



7 DISASTER RECOVERY

- Multi-Region Strategy
Active-Passive or Active-Active
- Route 53 Failover Routing
Automatic traffic switch
- S3 Cross-Region Replication
- Database Read Replicas
Cross-region replicas
- Infrastructure as Code
Rebuild anywhere, fast

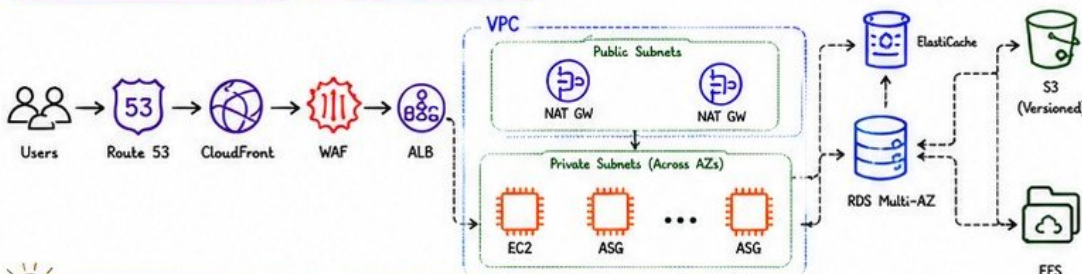
DR CHECKLIST

- ✓ Define RPO & RTO
- ✓ Automate failover
- ✓ Keep DR environment up to date
- ✓ Test DR plan regularly
- ✓ Document runbooks and contacts



Plan for the worst. Hope for the best.

8 PRODUCTION REFERENCE ARCHITECTURE (EXAMPLE)



KEY TAKEAWAYS

- ✓ Design for failure, not success
- ✓ Use managed services
- ✓ Automate & test everything
- ✓ Security by default
- ✓ Monitor, measure, improve

A good architecture keeps your users happy even when things break.



SENIOR ENGINEER TIP

Simplicity wins. Start simple, measure, then evolve. Over-designing is as bad as under-designing.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

SENIOR DEVOPS INTERVIEW SCENARIOS



REAL AWS PRODUCTION INTERVIEW QUESTIONS

★ Senior interviews test your **experience**, **judgment**, and **trade-off thinking** — not just tools. Think like an **owner**. Answer like a leader.

INTERVIEWER MINDSET

- ✓ Can you solve real problems?
- ✓ Can you make sound trade-offs?
- ✓ Do you understand ownership?
- ✓ Can you design & operate at scale?
- ✓ Do you learn and improve?



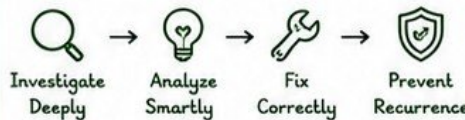
1 REAL SCENARIOS

- 1 An EC2 instance is down and users are impacted. How do you investigate?
- 2 CloudWatch alarm is not triggering for high CPU.
- 3 S3 bucket is not accessible from your application.
- 4 Your AWS bill suddenly increased by 300%. What do you do?
- 5 How would you design a highly available web application on AWS?
- 6 A deployment failed and rolled back. How do you find the root cause?
- 7 RDS database is slow. How do you troubleshoot?
- 8 You need to migrate a large on-prem application to AWS. How do you plan?



2 EXPECTED SENIOR-LEVEL ANSWERS

- ✓ Show a structured approach (not random actions)
- ✓ Explain your thought process clearly
- ✓ Use AWS services the right way
- ✓ Consider impact, trade-offs, and alternatives
- ✓ Think about monitoring, automation, and scale
- ✓ Communicate and document clearly
- ✓ Take ownership and drive resolution
- ✓ Prevent recurrence (not just fix the issue)



3 WHAT INTERVIEWERS LOOK FOR

- ✓ Problem solving & troubleshooting skills
- ✓ AWS service knowledge & depth
- ✓ System design & architecture thinking
- ✓ Trade-off analysis & cost awareness
- ✓ Monitoring, logging & observability
- ✓ Security & best practices
- ✓ Automation & IaC mindset
- ✓ Ownership, communication & leadership



TOP INTERVIEW QUESTIONS WITH APPROACH OUTLINE

1 EC2 INSTANCE IS DOWN



EC2

- Check status checks
- Review CloudWatch metrics
- Check system logs
- Verify networking & SG
- Check recent changes
- Identify root cause
- Implement fix & validate

2 ALARM NOT TRIGGERING



- Verify alarm configuration
- Check metric & namespace
- Review evaluation period
- Check "Treat missing data"
- Validate metric is publishing
- Test with manual data
- Fix & monitor

3 S3 ACCESS DENIED



- Check IAM permissions
- Review bucket policy
- Verify ACL settings
- Check VPC endpoint / route
- Validate region & endpoint
- Test with CLI / SDK
- Fix & re-test

4 COST SPIKE



- Open Cost Explorer
- Find top services
- Drill down by usage type
- Identify root cause
- Apply immediate controls
- Set budgets & alerts
- Optimize & automate

5 DESIGN HA WEB APP



- Multi-AZ VPC design
- ALB for load balancing
- Auto Scaling for compute
- RDS Multi-AZ (or Aurora)
- ElastiCache for caching
- S3 + CloudFront for static
- Monitoring + backups

6 DEPLOYMENT FAILED



- Check pipeline logs
- Review application logs
- Compare last good vs bad
- Check infra / dependency
- Reproduce in lower env
- Identify root cause
- Fix & improve pipeline

7 RDS DATABASE IS SLOW



- Check CloudWatch metrics
- Review slow query logs
- Analyze CPU, I/O, connections
- Check indexes & queries
- Review instance size
- Optimize & test
- Add monitoring & alerts

8 MIGRATE TO AWS



- Assess current environment
- Define target architecture
- Plan (lift & shift / refactor)
- Set up landing zone
- Migrate in phases
- Testing & cutover plan
- Optimize & secure

SENIOR ANSWER FRAMEWORK

- C** Clarify the question
- O** Outline your approach
- V** Validate assumptions
- E** Explain solution / actions
- R** Risks, trade-offs & alternatives
- E** Execute & monitor
- R** Review & prevent recurrence



EXAMPLE ANSWER STRUCTURE

- 1 Understand the problem
- 2 Gather information & data
- 3 Analyze & identify root cause
- 4 Take corrective action
- 5 Validate resolution
- 6 Prevent future occurrence



PRO TIPS

- ★ Share real examples from your experience
- ★ Use the STAR method (Situation, Task, Action, Result)
- ★ Communicate clearly & concisely
- ★ Think out loud, don't rush
- ★ Ask clarifying questions
- ★ Show ownership & follow-through



SENIOR ENGINEER TIP

Interviews are not about perfect answers.
They are about your thinking, experience, and leadership.



AWS PRODUCTION READINESS CHECKLIST



★ Use this checklist before going live and continuously in production.
Production is **earned**, not declared.

READINESS MINDSET

- ✓ Automate everything
- ✓ Monitor everything
- ✓ Secure everything
- ✓ Test everything
- ✓ Document everything
- ✓ Improve everything



1 INFRASTRUCTURE CHECKLIST

- ☐ VPC designed with public & private subnets
- ☐ Route Tables are correct and minimal
- ☐ Resources are in correct AZs
- ☐ No hardcoded values (use SSM / Secrets)
- ☐ IaC used and version controlled
- ☐ Resources are tagged consistently
- ☐ Backups enabled for critical services
- ☐ Patching strategy defined & automated
- ☐ Health checks configured
- ☐ Unused resources are removed



2 SECURITY CHECKLIST

- ☐ IAM least privilege applied
- ☐ MFA enabled for privileged users
- ☐ Strong password & rotation policies
- ☐ Security Groups follow least privilege
- ☐ NACLs configured (if required)
- ☐ Secrets stored in Secrets Manager / SSM
- ☐ Encryption at rest enabled (S3, RDS, EBS)
- ☐ Encryption in transit enforced (TLS)
- ☐ WAF enabled for public applications
- ☐ GuardDuty, Security Hub enabled
- ☐ Regular access reviews performed



3 MONITORING CHECKLIST

- ☐ CloudWatch metrics for all critical services
- ☐ Alarms configured with proper thresholds
- ☐ Alarms have actions (SNS / Lambda / etc.)
- ☐ Logs centralized (CloudWatch / S3)
- ☐ Dashboards created for key metrics
- ☐ X-Ray / tracing enabled (if applicable)
- ☐ Log retention policies configured
- ☐ Health checks and synthetic monitors set
- ☐ Alerts tested and validated
- ☐ Runbooks documented and accessible



4 COST CHECKLIST

- ☐ Cost Explorer reviewed and baselines set
- ☐ Budgets created with alerts
- ☐ Rightsize instances regularly
- ☐ Auto Scaling configured for efficiency
- ☐ Unused EBS / EIPs / Snapshots removed
- ☐ S3 lifecycle policies implemented
- ☐ Reserved Instances / Savings Plans used
- ☐ Spot / Graviton considered where possible
- ☐ Tagging for cost allocation in place
- ☐ Monthly cost review scheduled



5 HIGH AVAILABILITY CHECKLIST

- ☐ Multi-AZ deployment for critical services
- ☐ No single points of failure
- ☐ Auto Scaling enabled and tested
- ☐ Load Balancer health checks configured
- ☐ RDS Multi-AZ / Aurora used (if required)
- ☐ S3 used for shared & static data
- ☐ Cross-AZ data transfer minimized
- ☐ Failover tested and documented
- ☐ Dependencies are also highly available
- ☐ Capacity limits and quotas reviewed



6 DISASTER RECOVERY CHECKLIST

- ☐ RPO and RTO defined and documented
- ☐ Backups automated and tested
- ☐ Cross-Region replication enabled (S3 / RDS)
- ☐ AMI / IaC templates stored securely
- ☐ DR environment can be launched fast
- ☐ Route 53 failover or traffic shifting ready
- ☐ Data restore procedures tested
- ☐ DR runbook documented and rehearsed
- ☐ DR contacts and escalation plan ready
- ☐ Regular DR drills and improvements



QUICK PRODUCTION READINESS SUMMARY



Secure

Protect everything by default



Reliable

Design for failure, not success



Observable

You can't fix what you can't see



Efficient

Optimize cost continuously



Automated

Eliminate manual steps



Recoverable

Prepare for the worst, hope for best

REMEMBER



Production is a system, not a moment.
Readiness is a habit, not a checklist.
Keep improving!



VERIQTA PRODUCTION PRINCIPLES



1. OWNERSHIP

Take ownership end-to-end. You build it, you run it.



2. SIMPLICITY

Simple systems are easier to operate and scale.



3. AUTOMATION

Automate everything that can be automated.



4. OBSERVABILITY

If you can't observe it, you can't operate it.



5. SECURITY

Security is not a feature, it's a foundation.



6. COST AWARENESS

Every resource has a cost. Optimize relentlessly.



7. RESILIENCE

Build systems that can fail and recover gracefully.



8. CONTINUOUS IMPROVEMENT

Learn, iterate, and make things better every day.

FINAL CHECK

- ☐ All critical checklists reviewed
- ☐ Risks identified and mitigated
- ☐ Runbooks & docs are ready
- ☐ Team trained and aligned
- ☐ DR tested and validated



YOU ARE PRODUCTION READY!



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA